

MANUAL DE ESPECIFICACIONES TÉCNICAS: ARQUITECTURA DEL CORE BANCARIO, ORQUESTACIÓN CLOUD E INFRAESTRUCTURA DE SEGURIDAD PERIMETRAL

ENTIDAD RESPONSABLE: Neobank Digital, S.A.

MÓDULO 4: Core Bancario e Infraestructura Cloud (Arquitectura de Microservicios, API Gateway y Seguridad)

ENTREGABLE 4.1: Blueprint de Arquitectura de Microservicios, Gestión de Estado, Idempotencia y Orquestación Asíncrona (Documento 1 de 3)

VERSIÓN DE CONTROL: 1.0.0-TECHNICAL-ARCH

RESTRICCIÓN: Documento de Ingeniería de Software Mandatorio / Auditoría de Sistemas SIB

CAPÍTULO I: DEL DISEÑO TOPOLÓGICO DE MICROSERVICIOS DESACOPLADOS

1.1. Principios de Descomposición de Dominios Financieros

Neobank Digital, S.A. rechaza de forma categórica el despliegue de estructuras monolíticas tradicionales en su entorno transaccional. La topología de red y de cómputo de la Entidad se rige bajo el principio fundamental del diseño guiado por el dominio (Domain-Driven Design - DDD), subdividiendo los procesos de negocio en contextos delimitados (Bounded Contexts) que operan de forma autónoma y desacoplada a través de un clúster elástico gestionado por Kubernetes. Esta aproximación arquitectónica garantiza la tolerancia a fallos en caliente y el escalamiento horizontal independiente de cada componente, impidiendo que una degradación operativa en el módulo de consultas afecte el pipeline crítico de procesamiento de pagos.

Los subdominios nucleares del Core Bancario se encapsulan de forma estricta en microservicios independientes, aislados en su propia capa de red privada interna mediante políticas de red (NetworkPolicies) del plugin de CNI (Container Network Interface). Cada microservicio expone interfaces de programación de aplicaciones (APIs) fuertemente tipadas utilizando gRPC para la comunicación interna de alta velocidad y baja latencia, y REST/JSON para integraciones externas controladas a nivel de API Gateway. El principio de diseño exige la segregación total de responsabilidades y prohíbe que cualquier microservicio acceda de manera directa a la base de datos de un dominio ajeno, forzando la interacción exclusivamente por canales de mensajería o llamadas RPC autenticadas.

1.2. Mapeo Funcional de Microservicios Críticos del Core

La operación bancaria del neobanco se sustenta en una malla de microservicios distribuidos, los cuales interactúan de forma armónica para procesar transacciones financieras complejas en milisegundos. Para garantizar la trazabilidad de los flujos informáticos, las funciones del Core Bancario se dividen y distribuyen operativamente en los siguientes contenedores de cómputo inmutables:

- Microservicio de Cuentas y Saldos (Ledger Core Service):** Custodia el estado financiero en tiempo real de todos los depósitos monetarios y cuentas de ahorro de la Entidad. Es el único componente autorizado para ejecutar operaciones de débito o crédito directo en los registros contables primarios mediante transacciones ACID estrictas sobre el motor de base de datos relacional PostgreSQL.
- Microservicio de Clientes e Identidad (Identity & KYC Service):** Administra la información biográfica, firmas electrónicas, vectores de reconocimiento facial e historial del Formulario IVE de los usuarios. Interactúa directamente con el bus corporativo de integración que conecta al neobanco con el Registro Nacional de las Personas (RENAP).
- Microservicio de Transferencias ACH e Internacionales (Clearing House Service):** Orquesta el empaquetamiento, transformación estructural y firma criptográfica de los mensajes financieros destinados a la Cámara de Compensación Automatizada de Guatemala e integraciones de mensajería corresponsal vía red Swift de alta seguridad.
- Microservicio de Emisión y Control de Tarjetas (Card Lifecycle Service):** Gestiona el aprovisionamiento de tarjetas de débito físicas y virtuales Visa, el bloqueo de credenciales en caliente, la generación de números de tarjeta dinámicos para comercio electrónico y el procesamiento de mensajería basada en el estándar industrial ISO 8583.

CAPÍTULO II: DE LA GESTIÓN DE ESTADO DISTRIBUCIÓN Y PERSISTENCIA POLÍGLOTA

2.1. Arquitectura de Datos y Segregación de Responsabilidades (CQRS)

Para optimizar el rendimiento transaccional y asegurar que las operaciones de consulta masiva no degraden el rendimiento de los motores de escritura contable, Neobank Digital implementa de forma mandatoria el patrón arquitectónico de Segregación de Responsabilidades de Consulta y Comando (Command Query Responsibility Segregation - CQRS). Bajo este modelo, el almacenamiento de datos se bifurca operativamente en dos canales paralelos con características técnicas especializadas según su propósito de procesamiento:

La capa de comandos (Command Side) procesa exclusivamente mutaciones de estado que alteran saldos o crean cuentas. Utiliza un motor de base de datos PostgreSQL configurado con aislamiento de transacciones de nivel de lectura repetible (Repeatable Read) y replicación síncrona en alta disponibilidad hacia un nodo de respaldo. Por otro lado, la capa de consultas (Query Side) utiliza un clúster no relacional optimizado para lecturas masivas a gran velocidad (como MongoDB y Elasticsearch). La sincronización de datos entre ambas capas se ejecuta de forma asíncrona mediante un patrón de propagación de eventos controlado por un bus de datos empresarial, logrando una consistencia eventual garantizada en menos de doscientos (200) milisegundos a nivel global.

2.2. Matriz Operativa de Persistencia de Datos y Motores de Almacenamiento

La persistencia políglota de Neobank Digital asigna el motor de base de datos óptimo para cada dominio en función de su criticidad, requerimientos de consistencia matemática y velocidad de indexación, estructurándose de la siguiente forma corporativa:

Dominio de Negocio	Motor de Persistencia Seleccionado	Estrategia de Replicación y Respaldo	Nivel de Consistencia Operativa	Justificación Técnica de Cumplimiento
Contabilidad Central y Libro Mayor	PostgreSQL (Clúster de Alta Disponibilidad)	Replicación física síncrona en tres zonas de disponibilidad con backups por WAL continuos.	Consistencia Inmediata y Estricta (Cumplimiento ACID absoluto).	Garantiza la inmutabilidad de los saldos bancarios y previene la pérdida de transacciones contables ante caídas del clúster de cómputo.
Historial de Sesiones y Token de Autenticación	Redis (Clúster In-Memory con Persistencia AOF)	Replicación maestro-esclavo con conmutación por error automatizada vía Redis Sentinel.	Consistencia Eventual de alta velocidad.	Provee accesos a datos de validación de tokens de sesión con latencias inferiores a un (1) milisegundo en picos de demanda.
Auditoría Forense y Monitoreo de Eventos Logísticos	Elasticsearch / OpenSearch Clúster	Fragmentación (Sharding) de datos por mes calendario con réplicas cruzadas en caliente.	Consistencia Eventual.	Permite la indexación de millones de logs y telemetría de red para análisis forenses e investigaciones de cumplimiento AML rápidas.
Catálogo de Comercios y Preferencias del Usuario	MongoDB Atlas (Documental)	Conjunto de réplicas distribuidas geográficamente con copias instantáneas (Snapshots) diarias.	Consistencia Eventual Controlada.	Soporta esquemas altamente dinámicos y mutables que cambian según las campañas comerciales sin requerir paradas de base de datos.

CAPÍTULO III: DE LOS MECANISMOS DE IDEMPOTENCIA Y PROCESAMIENTO SEGURO DE PAGOS

3.1. Mitigación del Riesgo de Doble Cargo y Colisiones de Red

En un entorno financiero digital propenso a intermitencias de red celular y caídas temporales de enlaces de datos, el procesamiento repetido de una misma transacción representa uno de los riesgos operacionales más severos para la estabilidad contable y la experiencia del cliente. Para anular este vector de falla, Neobank Digital integra un mecanismo de idempotencia estricto en la capa de ingreso del Core Bancario. La idempotencia garantiza que el reenvío de una solicitud de pago idéntica por parte del dispositivo móvil del usuario (debido a retardos en la recepción de la respuesta original) no genere cargos duplicados en la cuenta de ahorros del cliente.

El procedimiento operativo exige que todo intento de mutación financiera incluya obligatoriamente una llave de idempotencia única (Idempotency-Key) generada en el front-end de la aplicación móvil utilizando el formato estandarizado UUID versión 4. Al recibir la solicitud, el componente Middleware intercepta el paquete y consulta el clúster de caché en memoria de Redis empleando una operación atómica. Si la llave de idempotencia no existe en el registro, el sistema la almacena inmediatamente con un estado **PROCESANDO** y un tiempo de vida (TTL) configurado de forma mandatoria en setenta y dos (72) horas calendario, permitiendo el pase de la transacción hacia el Core contable para su procesamiento ordinario.

3.2. Ciclo de Vida del Registro de Claves de Idempotencia

Si un reenvío de red introduce una llave de idempotencia que ya existe en el registro de Redis con estado **PROCESANDO**, el sistema bloquea inmediatamente la ejecución concurrente del hilo, descartando la colisión transaccional y devolviendo un código HTTP 409 Conflict. Cuando el microservicio original concluye de forma exitosa la escritura contable en PostgreSQL, el estado de la llave en Redis muta a **COMPLETADO** e incrusta el payload exacto de la respuesta bancaria generada.

Todo intento subsiguiente de consumir el API con esa misma llave no tocará los motores de base de datos relacionales, sino que recuperará directamente la respuesta almacenada en la caché de Redis y la entregará al front-end en microsegundos, emulando un procesamiento exitoso sin duplicar el flujo financiero. El ciclo de vida operativo de las claves de idempotencia se rige bajo los siguientes estados inmutables:

1. **Estado INICIADO:** Registrado al momento de recibir el encabezado HTTP 'Idempotency-Key' válido y superar las reglas de firma digital perimetrales.
2. **Estado EJECUCION_LEDGER:** Momento en el cual el hilo de ejecución abre la transacción contable transaccional dentro del motor PostgreSQL. Los recursos financieros quedan reservados preventivamente.
3. **Estado RESOLVIDO_EXITO:** Mutación del registro tras el cierre exitoso del bloque contable. Almacena los códigos de confirmación de transferencia ACH o autorización Visa.
4. **Estado ERROR_REVERTIDO:** Activado si la transacción contable falla por fondos insuficientes o parámetros inválidos. Libera la clave para permitir intentos corregidos del usuario bajo un UUID nuevo.

CAPÍTULO IV: DEL MOTOR DE ORQUESTACIÓN ASÍNCRONA Y COREOGRAFIA DE EVENTOS

4.1. Configuración del Bus de Mensajería de Alta Resiliencia

La comunicación asíncrona entre los microservicios de Neobank Digital se instrumenta sobre una infraestructura de bus de datos empresarial basada en Apache Kafka, desplegada en topología de clúster con factor de replicación tres (3) distribuido de manera uniforme entre las zonas de disponibilidad de la nube. Kafka opera como la columna vertebral de eventos (Event Backbone) de la institución, gestionando de forma inmutable los flujos de mensajería que notifican las actividades de los usuarios, las alertas del motor de fraude y los cambios de estado en las tarjetas de débito.

Cada tipo de evento crítico de negocio se mapea de forma estricta en un tópico específico de Kafka (por ejemplo, **transaction.ledger.journal** para movimientos contables o **customer.kyc.approved** para onboarding exitosos). Las políticas de configuración del clúster prohíben la creación automática de tópicos y exigen que la persistencia se parametrize con acuse de recibo completo de todos los nodos (**acks=all**), garantizando de forma absoluta que un mensaje no sea considerado como entregado hasta que haya sido escrito de forma segura en los discos de almacenamiento de todos los agentes (Brokers) de respaldo del clúster, impidiendo la pérdida de pistas de auditoría financiera.

4.2. Implementación de Patrones Saga para Transacciones Distribuidas

Dado que los dominios financieros se encuentran fragmentados en múltiples microservicios con bases de datos independientes, el neobanco no puede recurrir a las transacciones de base de datos locales tradicionales para asegurar la

consistencia entre dominios cruzados. Para resolver esto de forma operativa, se implementa el patrón arquitectónico Saga en modalidad de Orquestación para transacciones complejas multinodo (como el flujo de alta de usuario con emisión inmediata de tarjeta virtual y fondeo inicial).

Un orquestador central basado en Camunda BPMN gestiona la máquina de estados de la Saga. Si el usuario solicita la apertura de una cuenta Nivel 3 y el fondeo por tarjeta externa, el orquestador invoca de forma secuencial los servicios correspondientes emitiendo comandos en los tópicos de Kafka. El procedimiento operativo exige el diseño mandatorio de Acciones Compensatorias para cada paso de la Saga; si el servicio de cuentas crea el registro de balances con éxito, pero el servicio de tarjetas falla catastróficamente al intentar aprovisionar el plástico virtual en los sistemas de Visa, el orquestador intercepta el código de fallo y dispara en reversa la secuencia de compensación, ejecutando llamadas de anulación contable para regresar los fondos del cliente a su estado original, asegurando que el sistema no quede jamás en un estado inconsistente o de limbo regulatorio.

CAPÍTULO V: DEL PIPELINE AUTOMATIZADO DE ENTREGA CONTINUA (CI/CD)

5.1. Políticas de Inmutabilidad del Software y Pruebas Estáticas

El despliegue de código fuente en los entornos productivos de Neobank Digital está estrictamente prohibido para el personal de ingeniería de forma manual o directa. Toda modificación de software, corrección de errores en microservicios o alteración de parámetros del Core Bancario debe someterse de forma mandatoria a un flujo de trabajo controlado y automatizado gestionado por un Servidor de Integración Continua (como GitLab CI/CD o GitHub Actions), asegurando la inmutabilidad de los artefactos tecnológicos antes de su inyección en producción.

El pipeline de CI/CD se activa automáticamente tras la aprobación de un pull request dirigido a la rama principal (*main*), requiriendo la firma criptográfica y validación manual de al menos dos arquitectos de software Senior (Regla de Cuatro Ojos). La primera fase del pipeline ejecuta herramientas de análisis estático de código (SAST) mediante instancias integradas de SonarQube, bloqueando el avance de la compilación si se detecta una cobertura de pruebas unitarias inferior al ****noventa y cinco por ciento (95%)**** o si se identifican vulnerabilidades críticas de seguridad, tales como contraseñas quemadas en el código fuente o inyecciones de dependencias de terceros maliciosas.

5.2. Procedimiento Operativo de Despliegue Progresivo Canary

Una vez compilada la imagen de contenedor Docker y firmada digitalmente con una llave privada institucional mediante Cosign, el software pasa a la fase de despliegue controlado en el clúster de Kubernetes productivo. La plataforma implementa de forma mandatoria la estrategia de despliegue Canary gestionada por la herramienta de entrega progresiva Argo Rollouts, minimizando el radio de explosión operativa ante la introducción de código defectuoso que haya evadido los entornos de control previos.

El procedimiento operativo de despliegue Canary se ejecuta siguiendo una secuencia temporal rigurosamente parametrizada y automatizada:

- 1. Fase de Enrutamiento Inicial (Canary 5%):** Argo Rollouts despliega una única instancia (Pod) del nuevo microservicio y modifica las reglas del controlador de ingreso para desviar únicamente el **cinco por ciento (5%)** del tráfico de usuarios reales hacia la nueva versión, manteniendo el noventa y cinco por ciento restante en la versión estable previa.
- 2. Fase de Evaluación de Telemetría Crítica (Ventana de 30 Minutos):** Durante este intervalo de tiempo, el sistema de monitoreo (Prometheus) evalúa de forma continua los indicadores clave de rendimiento (KPIs) de la nueva versión. La tasa de errores HTTP 5XX debe permanecer en ****cero por ciento (0%)**** y el percentil 99 de latencia de las transacciones (P99) debe ser inferior a ****ciento cincuenta (150) milisegundos****.
- 3. Fase de Escalamiento Escalonado (Canary 25% y 50%):** Si la telemetría valida la estabilidad del software, el sistema eleva de forma automática el enrutamiento de tráfico al veinticinco por ciento (25%) y posteriormente al cincuenta por ciento (50%), repitiendo las rutinas de validación de métricas de rendimiento en cada hito.
- 4. Fase de Promoción Total (Rollout 100%):** Superadas las fases previas sin anomalías de sistema, la nueva versión se promueve como la versión oficial de producción, procediendo al apagado controlado de los contenedores antiguos. Ante la menor desviación detectada por Prometheus en cualquier fase, el sistema ejecuta de forma autónoma e instantánea un desvío de reversa (Automated Rollback) al cien por ciento a la versión anterior en menos de un segundo, notificando de urgencia al equipo de operaciones de seguridad (DevSecOps).